



MERRIMACK COLLEGE

CSC 6303

Week 3

From Python to Go, Swift, and modern languages

Systems and Languages Survey - Dr. Paulo Fernandes

Presentation Agenda

Week 3

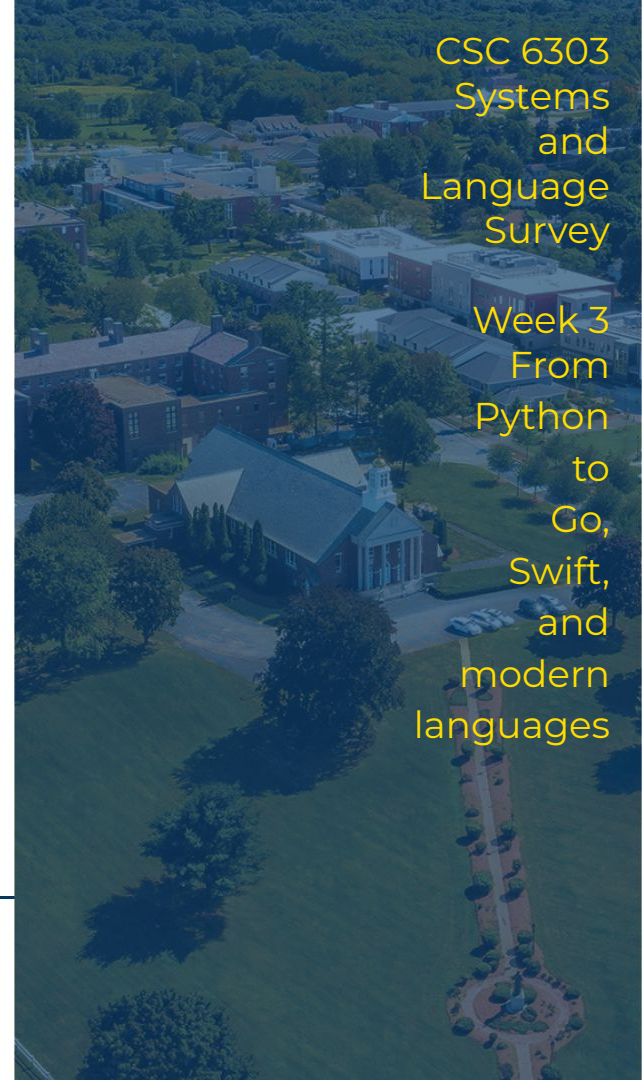
- Using Go and Swift
 - a. Online or With and IDE
- Go from Google
 - a. Purpose
 - b. Basic Commands
 - c. OO and Go
- Swift from Apple
 - a. Purpose
 - b. Basic Commands
- This Week's tasks



MERRIMACK COLLEGE

CSC 6303
Systems
and
Language
Survey

Week 3
From
Python
to
Go,
Swift,
and
modern
languages



Using Go and Swift

You can use it online or have it installed in your own machine available on an IDE

Likely your machine hasn't either Go or Swift installed. While Go is fairly easily installed in any machine, Swift can only be easily installed in Mac OS and Linux, in Windows the process is trickier.

This aspect reflects the difference of approach between those two languages, despite their common purpose to serve modern systems programming needs.



Running code online

Among the several online options you can try these ones:

You can try it on one of the following
Go interpreters online:

[Better Go Playground](#) - [Replit](#) - [JDoodle](#)

You can try it on one of the following
Swift interpreters online:

[Replit](#) - [JDoodle](#) - [Swiftfiddle](#)

A curious thing is that such online servers always behave as interpreters, but refer themselves as compilers, probably due to their linking functionalities.



MERRIMACK COLLEGE

Friendly advice, using an online server as such, always
keep a local copy of your code.

Installing Go and Swift in your machine

- Choose an IDE
 - There are several options, I personally like Visual Studio Code that is free, available to all Operating Systems, and reasonably light compared to other options
 - VSCode official page: <https://code.visualstudio.com>
- Install the proper compiler according to your Operating System (Windows, MacOS, or Linux)
 - Walkthrough for the setup of Go language in Visual Studio [here](#)
 - Setting up Swift in a Windows system with VSCode Marketplace [here](#)
 - Setting up Swift in a MacOS system [here](#)



These languages are still evolving, so are the running environment for them.

Go from Google

A modern language to rule them all, or at least, what the language designers intended to.

The basic idea of Google when creating Go was to integrated existent languages concepts, as well as to offer easier primitives for common demands in modern applications.

The Go language (Golang) purpose is to respond to new needs, while providing a safer development environment. As such it is a far stretched, ambitious bet.

- Purpose
- Basic Commands
- OO and Go



History and Purpose



Created in 2009 within Google to be as efficient as C, as easy as Python, and do things right where C++ did wrong.

The basic goals of Go language are to provide a lighter syntax to answer modern demands and at the same time processing safeguards to avoid sloppy code.

- Modern demands:
 - Web development, Cloud and Network services:
 - fast and scalable packages, plus APIs for providers;
 - Command-line interfaces, DevOps and site reliability:
 - standard library, syntax support, and doc generator;
- To avoid sloppy code:
 - Typed language, index verification, lean syntax;
 - The wealth of Google tools to support the language.



Program Structure

- In **Python**
 - Mostly packages, sometimes scripts, sometimes a main function, sometimes classes
- In **Go**
 - Packages, a main function calling everything else, and methods (without classes)

```
// Bubblesort implementation in Golang
package main
import "fmt"

func bubbleSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        for j := 0; j < n-i-1; j++ {
            if arr[j] > arr[j+1] {
                arr[j], arr[j+1] = arr[j+1], arr[j]
            }
        }
    }
}

func main() {
    arr := []int{64, 34, 25, 12, 22, 11, 90}
    bubbleSort(arr)
    fmt.Println("Sorted array is:")
    fmt.Println(arr)
}
```

C++
similar
style
{ }



No ;



Basic Commands - functions

- In Python - **def**
 - functions are untyped
 - input parameters can be omitted/default values
 - simple variables passed by value, lists passed by reference (almost always)
 - none, single, or multiple untyped return
- In Go - **func**
 - functions are explicitly typed if returning something
 - input parameters passed either by value or reference (programmer choice to use pointers)
 - none, single, or multiple typed return
 - (no function overload on different parameter sets)

```
def sumMinMax(a, b):  
    sum = a + b  
    if a < b:  
        min, max = a, b  
    else:  
        min, max = b, a  
    return sum, min, max
```



```
func sumMinMax(a, b int) (sum, min, max int) {  
    sum = a + b  
    if a < b { min, max = a, b }  
    else { min, max = b, a }  
    return  
}
```



pass by reference



fillCup()

pass by value



fillCup()

www.penjee.com



MERRIMACK COLLEGE

Text: [Golang Pass by value vs Pass by reference.](#)

Basic Commands - variables

- In Python
 - Integers, Floating points, Booleans, and Lists (including Strings)
 - No declaration or allocation needed
- In Go
 - Integers, Floating points, Complex numbers (several precisions), Booleans, and Strings
 - Typed declaration needed or inferred
 - **var**
 - within **func** parameters
 - using the **:=** constructor



```
var found bool = false
var s      string = "example"
var i      int = 0    // int8  byte  int16  int32  rune  int64
var acc    uint = 1   // uint8 uint16 uint32 uint64 uintptr
var x, y   float32    // float64
var z      complex64 // complex128
```

```
func absDiff(a, b float64) float64 {
    diff := a - b
    if diff < 0
        return -diff
    else
        return diff
}
```



Basic Commands - decision

- In Python
 - if - elif - else
- In Go
 - if - else

```
if diff < 0
    return -diff
else
    return diff
```

```
if v <= 100 {
    fmt.Printf("up to 100\n")
} else if v <= 200 {
    fmt.Printf("up to 200\n")
} else if v <= 500 {
    fmt.Printf("up to 500\n")
} else {
    fmt.Printf("above 500\n")
}
```

```
if a < b { min, max = a, b }
else { min, max = b, a }
```

```
switch s[0] {
case "C":      fmt.Println("C#")
case "g", "G": fmt.Println("Go")
case "j", "J": fmt.Println("Java")
default:      fmt.Println("Python")
}
```

```
if expression1:
    statement(s)
elif expression2:
    statement(s)
elif expression3:
    statement(s)
else:
    statement(s)
```



```
var variable interface{}
switch q := variable.(type) {
case bool:
    fmt.Println("variable is boolean")
case float32, float64:
    fmt.Println("variable is float")
case int, byte, int8, int16, int32, int64,
uint, rune, uint8, uint16, uint32, uint64, uintptr:
    fmt.Println("variable is int")
default:
    fmt.Printf("value is of type: %T", q)
}
```



- switch - case - default
 - expression and type



Basic Commands - loops

- In Python
 - for loop - range - break - continue - pass - else
 - while loop - break - continue - else
- In Go - **for**
 - *traditional for loops*
 - *infinite loops*
 - *conditional loops*
 - range in loops
 - string loops
 - maps loops
 - channel loops

```
fruits = ['banana', 'apple', 'mango']
for index in range(len(fruits)):
    print 'Current fruit :', fruits[index]
```

```
count = 0
while (count < 9):
    print 'The count is:', count
    count = count + 1
```



```
// similar to C++ or Java for loop
for i := 0; i < 10; i++ {
    fmt.Printf("Iteration ", i)
}
```

```
// similar to Python while True
i := 0
for {
    if i == 10 { break }
    else { fmt.Printf("Iteration ", i) i++ }
}
```

```
// similar to Python while loops
i := 0
for i < 10 {
    fmt.Printf("Iteration ", i)
    i++
}
```



Basic Commands - loops

- In Python
 - for loop - range - break - continue - pass - else
 - while loop - break - continue - else
- In Go - **for**
 - traditional for loops
 - infinite loops
 - conditional loops
 - *range in loops*
 - *string loops*
 - maps loops
 - channel loops

```
fruits = ['banana', 'apple', 'mango']  
for index in range(len(fruits)):  
    print 'Current fruit :', fruits[index]
```

```
count = 0  
while (count < 9):  
    print 'The count is:', count  
    count = count + 1
```



```
// similar to Python list loops  
myList:= []string{"first", "second", "third", "forth"}  
  
for index, element:= range myList {  
    | fmt.Println(index, element)  
}
```

```
// similar to Python string loops  
for index, element:= range "abcdef" {  
    | fmt.Println(index, character)  
}
```



Basic Commands - loops

- In Python
 - for loop - range - break - continue - pass - else
 - while loop - break - continue - else
- In Go - **for**
 - traditional for loops
 - infinite loops
 - conditional loops
 - range in loops
 - string loops
 - *maps loops*
 - *channel loops*

```
fruits = ['banana', 'apple', 'mango']
for index in range(len(fruits)):
    print 'Current fruit :', fruits[index]
```

```
count = 0
while (count < 9):
    print 'The count is:', count
    count = count + 1
```

```
// similar to Python dictionary loops
myMap := map[int]string{
    1:"first",
    3:"third",
    10:"tenth",
    100:"hundredth"
}
for key, data:= range myMap {
    fmt.Println(key, data)
}
```

```
// similar to C++ streams
myChannel := make(chan int)
go func(){
    myChannel <- 1
    myChannel <- 3
    myChannel <- 10
    myChannel <- 100
    close(myChannel)
}()
for i:= range chnl {
    fmt.Println(i)
}
```

More info about [maps](#) and [channels](#)



MERRIMACK COLLEGE

Reading about Go loops from [Geeks for Geeks](#).

Object in Go

Go does not have classes, but user defined data structures and methods associated to it

- A user defined data structure with the reserved words **type** and **struct**
- A method is a function that is associated to a type (just between the reserved word **func** and the function name)
 - An assessor method can receive the type instance by value, as computing the area of a rectangle
 - A mutator method has to receive the type instance by reference, as setting the dimensions of a rectangle



```
package main

import ("fmt")

type Rectangle struct {
    length int
    width  int
}

func (r Rectangle) Area() int {
    return r.length * r.width
}

func (r *Rectangle) SetDimensions(l, w int) {
    r.length = l
    r.width = w
}

func main() {
    rect := Rectangle{length: 3, width: 4}
    fmt.Println("Rect area: ", rect.Area())
    (&rect).SetDimensions(5, 6)
    fmt.Println("Rect area: ", rect.Area())
}
```



More on Go

- Go offers concurrency features using goroutines with the reserved word **go**
 - Channels are used to communicate among subroutines
- Go provides several built-in services as:
 - documentation similar to *javadocs* and *pydoc*, called **godoc**
 - installation of packages similar to Python's *pip* called **go install**
 - automatic module creation with **go mod**
 - compiler safe variable renaming with **gorename**
 - unit testing with **go test**
 - static code analysis for debugging with **go vet**
- Go is criticized by trying to bridge:
 - light syntax as Python (for example multiple assignments) and
 - coding safeguards as C++ (typed variables, for example)



Swift from Apple

A language to push the production of software for Apple products, much closer to C than Go.

The basic idea of Apple when creating Swift was to integrated existent languages concepts, as well as to offer easier primitives for common demands in Apple products' applications.

The Swift language has evolved since the creation, expanding the availability to other systems and producing code to other platforms.

- Purpose
- Basic Commands



History and Purpose

Created in 2014 within Apple to be a replacement of Objective-C, a C extension with OO concepts dating back to 80's, just like C++, but slower to evolve.



Objective-C was developed by NeXT company for their Operating System OpenStep, while Steve Jobs was the CEO of NeXT. Once Apple bought NeXT the new Mac OS X was based on the evolution of OpenStep, thus coded in Objective-C. As the needs evolve within Apple in 2014 the Swift language was introduced and nicknamed Objective-C without C. It was intended to respond to Apple's needs in terms of:

- Prevent some common bugs found in programming languages, as null-pointer dereferencing.
- Provide more reliable code using assertions and protocols.



Program Structure

- In **Python**
 - Mostly packages, sometimes scripts, sometimes a main function, sometimes classes
- In **Swift**
 - Packages, a main function calling everything else, and frequently classes

```
// Bubblesort in Swift
func bubbleSort(_ array: inout [Int]) {
    for i in 0..
```



C++
similar
style
{ }

```
var numbers = [64, 34, 25, 12, 22, 11, 90]
bubbleSort(&numbers)
print(numbers)
```

Optional ;



Basic Commands - functions

- In Python - **def**
 - functions are untyped
 - input parameters can be omitted/default values
 - simple variables passed by value, lists passed by reference (almost always)
- In Swift - **func**
 - functions are explicitly typed if returning something
 - input parameters typed passed either by value or reference (depending on the kind)
 - function overload on different parameter sets

```
def sumMinMax(a, b):  
    sum = a + b  
    if a < b:  
        min, max = a, b  
    else:  
        min, max = b, a  
    return sum, min, max
```



```
func sumMinMax(a: Int, b: Int) -> (sum: Int, min: Int, max: Int) {  
    var sum: Int  
    var min: Int  
    var max: Int  
    sum = (a + b)  
    if a < b {  
        min = a  
        max = b  
    }  
    else {  
        min = b  
        max = a  
    }  
    return (sum, min, max)  
}
```



Basic Commands - constants & variables

- In Python - untyped
 - Integers, Floating points, Booleans, and Lists (including Strings)
 - No declaration or allocation needed
- In Swift - type safe
 - Integers, Floating points (several precisions), Booleans, Strings, Optional, Tuples (like lists)
 - **var** for variables
 - **let** for constants
 - **enum** for enumerations
 - **typealias** for aliases
 - **struct** for data structures (or **class** for objects)



```
func isEven(number: Int) -> Bool {
    return (number % 2) == 0
}

func isOdd(number: Int) -> Bool {
    return (number % 2) == 1
}

func onlySomething(list: [Int], condition: (Int) -> Bool) -> Bool {
    for item in list {
        if !condition(item) {
            return false
        }
    }
    return true
}

let n1 = [51, 21, 56, 13, 11]
print(onlySomething(list: n1, condition: isEven))
var n2 = [51, 21, 55, 13, 11]
print(onlySomething(list: n2, condition: isOdd))
```



Basic Commands - decision

- In Python
 - if - elif - else
- In Swift
 - if - else
 - conditional operator



```
var freezing: Int
var scale: string
scale = "Celsius"
freezing = (scale == "Celsius") ? 0 : 32
```

- switch - case - default
 - fallthrough (break is the default)

```
var answer: string
switch answer {
    case "A" :
        print("Great")
    case "B" :
        print("Good")
    case "C" :
        print("Ok")
    case "D","F" :
        print("Ouch!")
    default :
        print("What?!")
}
```

```
if expression1:
    statement(s)
elif expression2:
    statement(s)
elif expression3:
    statement(s)
else:
    statement(s)
```



```
var freezing: Int
var scale: string
scale = "Fahrenheit"
if scale == "Fahrenheit"
{ freezing = 32 }
else if scale == "Celsius"
{ freezing = 0 }
else
{ freezing = -273 }
```



Basic Commands - loops

- In Python
 - for loop - range - break - continue - pass - else
 - while loop - break - continue - else
- In Swift
 - for in loops
 - while loops
 - repeat while loops
 - break - continue



```
fruits = ['banana', 'apple', 'mango']
for index in range(len(fruits)):
    print 'Current fruit :', fruits[index]
```

```
count = 0
while (count < 9):
    print 'The count is:', count
    count = count + 1
```



```
let values = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
for v in values {
    print(v)
}
```

```
var i = 0
while i < 10 {
    print(i)
    i += 1
}
```

```
var i = 0
repeat {
    print(i)
    i += 1
} while i < 10
```



Object Oriented Concepts in Swift

Object Oriented concepts in Swift are similar to Python by:

- Definition of subclasses for inheritance;
- Definition of methods and constructors;
- Definition of instance variables.

However, they differ in terms of:

- Methods overloading;
- Polymorphism with functions passed as parameters;
- Explicitly override of superclass methods.



```
class Person {
    var name: String
    var familyName: String

    init(name: String, familyName: String) {
        self.name = name
        self.familyName = familyName
    }

    func card() -> String {
        return self.name+" "+self.familyName
    }
}

class Student : Person {
    var major: String

    init(name: String, familyName: String, major: String) {
        self.major = major
        super.init(name: name, familyName: familyName)
    }

    override func card() -> String {
        return self.name+" "+self.familyName+" "+self.major
    }
}
```



This Week's tasks

- Post discussion
- Programming project #3

Tasks

- What do you believe are the advantages of using Go instead of Python?
- **Convert** a simple **Python** program into **Go**.



Post Discussion - Week 3 - In-class exercises

What do you believe are the advantages of using Go instead of Python?

- Python is the dominant programming language as of now (legacy code set aside), but Go has some novelties in terms of modern concepts and the might of Google to support it.
 - Feel free to give personal reasons, but relate those with technical characteristics of the language.

Your task:

- Post your discussion in the message board until this Saturday;
- Reply posts of your colleagues in the message board until next Wednesday.



MERRIMACK COLLEGE

This task counts towards the In-Class Exercises grade.

Progr. Project #3 - Week 3 - P#3

Convert this [Python program](#) into a Go lang.

A program conversion must reproduce the exact behavior, i.e., the same input in the original and convert programs must deliver the same output.

Run both the original and convert program before submitting!

```
def tribonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]
    else:
        fib_seq = [1, 1, 1]
        for i in range(3, n):
            next_num = fib_seq[i-1] + fib_seq[i-2] + fib_seq[i-3]
            fib_seq.append(next_num)
        return fib_seq

print(tribonacci(20))
```



Progr. Project #3 - Week 3 - P#3

- You must submit the Go code file and all accompanying files in a zipped folder.
- If you need to add any explanation or messages to me, make it in the code remarks.
- The quality of your code remarks is not relevant for the evaluation, but make sure you write your name in the remarks.
- This Python code of the program tribo.py is available [here](#).

Your task:

- Save your program code and submit it in the appropriate Canvas until Next Wednesday.

This assignment counts towards
the Coding Projects grade.



MERRIMACK COLLEGE

” Week 3 of CSC 6303

- **Post in the discussions until Saturday, reply until next Wednesday**
- **Do the coding project until next Wednesday**

Next Week - From Python to JavaScript and Perl



MERRIMACK COLLEGE